

PHASE 1 & 2 COMPLETED

Spendly: Backend Master Architecture

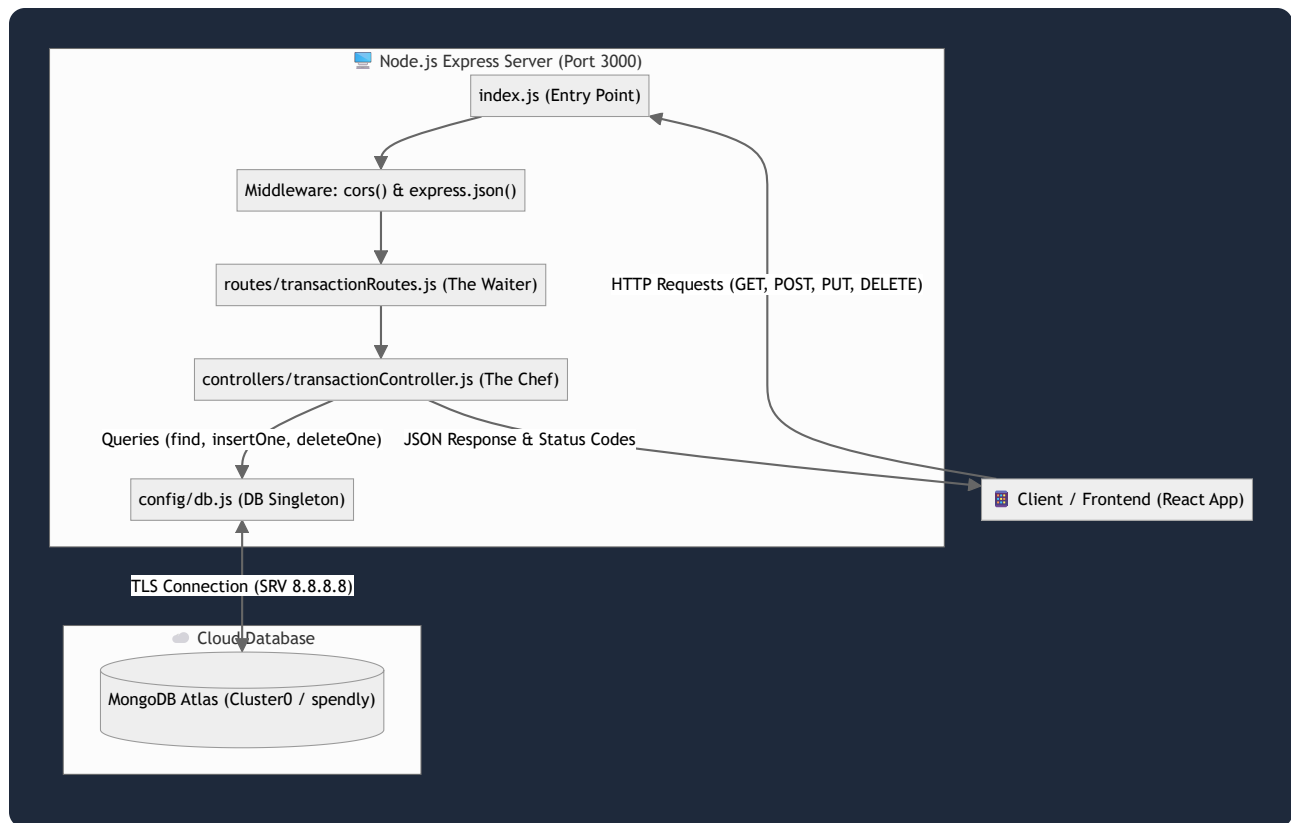
A complete conceptual and technical guide covering Express, MongoDB, RESTful design, validation guards, and data flow.



1. High-Level Architecture

Spendly is designed using a clean, layered architecture where each file has a single, well-defined responsibility.

SYSTEM LAYER FLOW



index.js (Entry Point)

Loads environment variables, initializes Express, attaches global middleware (CORS, JSON body parser), and mounts routes.



routes/ (The Waiter)

Maps incoming HTTP methods and URL paths (e.g. `POST /transactions`) directly to controller functions.



controllers/ (The Chef)



config/db.js (The Connection)

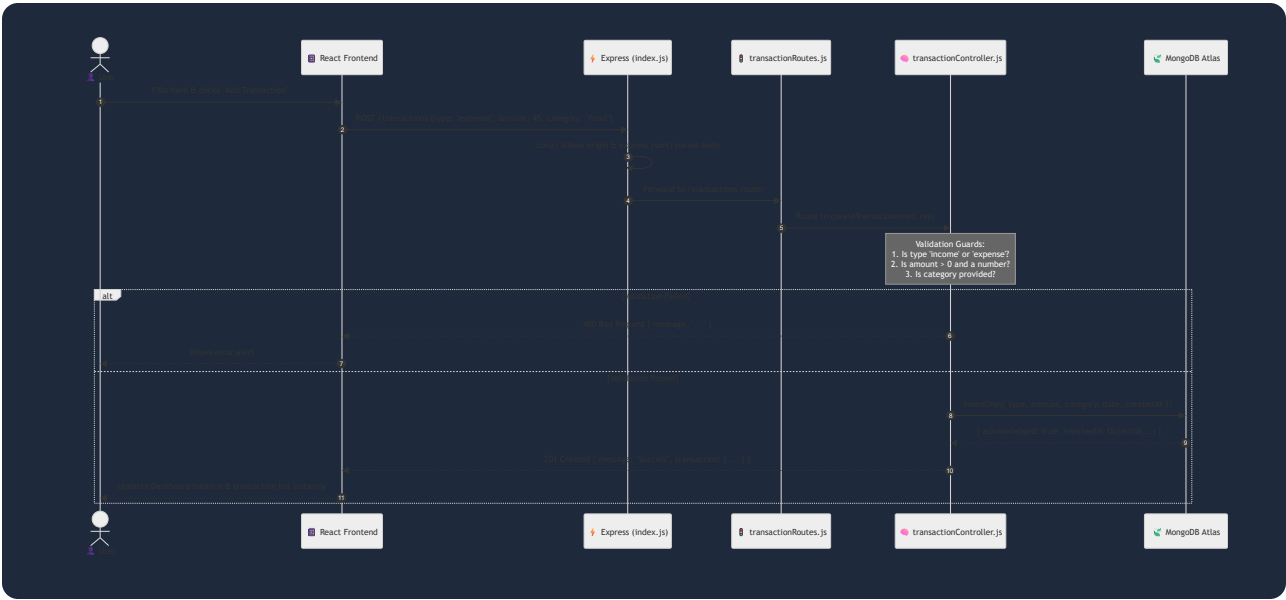
Contains all business logic: runs input validation guards, computes math, and talks to MongoDB.

Establishes and shares a single persistent connection pool to MongoDB Atlas across the server.

 2. Detailed Request-Response Lifecycle

Here is what happens step-by-step when a user logs a new expense (\$45 for Lunch):

SEQUENCE DIAGRAM: POST /TRANSACTIONS

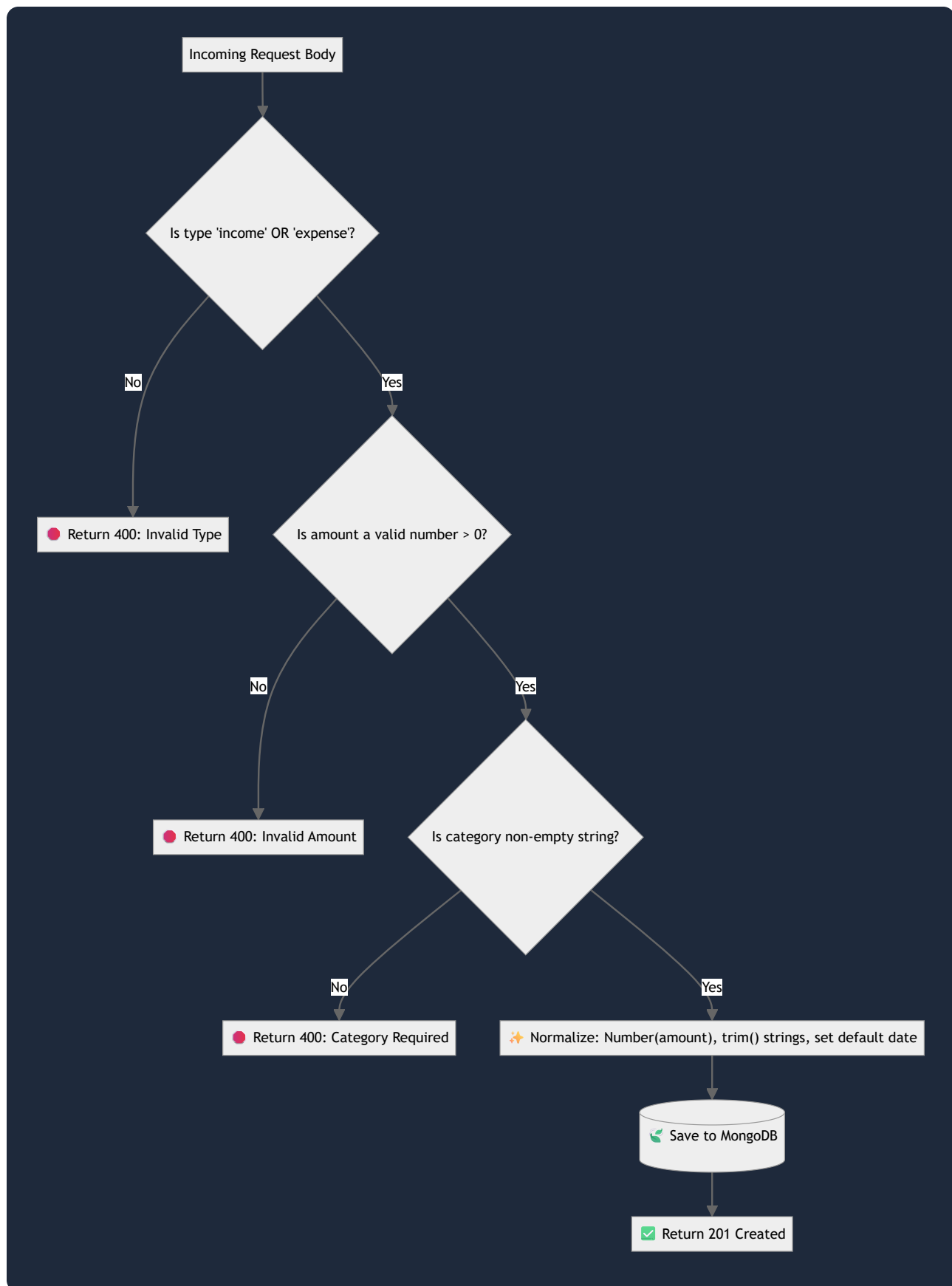




3. Input Validation & Guard Clauses

In professional backend engineering, we use the **Guard Clause Pattern** (checking for bad input first and rejecting early) to keep our database clean.

VALIDATION DECISION TREE



💡 Why did we use `&&` instead of `||` ?

```
if (!type || (type !== 'income' && type !== 'expense'))
```

If we used `||`, the expression would always evaluate to true for every string in the universe, because a word can never be both `'income'` AND `'expense'` at the same time. Using `&&` correctly checks that the input is neither one.



4. MongoDB Model & ObjectId Mechanics

In MongoDB, data is stored in JSON-like documents inside collections:

TRANSACTION DOCUMENT SCHEMA

TransactionDocument

```
+ObjectId _id : "65f1a2b3c4d5e6f7a8b9c0d1"  
+String type : "income" | "expense"  
+Number amount : 1250.00  
+String category : "Salary" | "Food" | "Rent"  
+String date : "2026-09-01"  
+String description : "Monthly paycheck"  
+Date createdAt : 2026-09-01T14:30:00.000Z
```

🔑 Why `new ObjectId(id)` is Required

MongoDB's `_id` is stored as a 12-byte binary BSON type, not a plain JavaScript string. If you query MongoDB with `{ _id: "65f..." }` as a string, MongoDB will never find the match. We must convert it using `new ObjectId(id)` and always check `ObjectId.isValid(id)` first to avoid runtime crashes.



5. REST API Endpoints Reference

METHOD	ENDPOINT	PURPOSE	SUCCESS STATUS	ERROR STATUS
GET	/transactions/summary	Calculates total income, total expense, and balance	200 OK	500 Server Error
GET	/transactions	Fetches array of all transactions	200 OK	500 Server Error
POST	/transactions	Creates a new income or expense	201 Created	400 Bad Request
PUT	/transactions/:id	Updates existing transaction fields	200 OK	400 / 404 / 500
DELETE	/transactions/:id	Deletes a transaction by its MongoDB ID	200 OK	400 / 404 / 500



6. Next Up: Phase 3 (React + Vite Frontend)

Now that the backend is complete, robust, and tested with MongoDB, here is how we will build the user interface:

FRONTEND COMPONENT HIERARCHY

